LAPORAN 4
ANALISIS KOMPLEKSITAS ALGORITMA

DISUSUN OLEH :

DESTYANA SILVIRA KURNIA
NIM.2025573010126

Kelas : TI 1C
Mata Kuliah : Algoritma dan Struktur Data
Jurusan : Teknologi Informasi dan Komputer
Prodi : Teknik Informatika
Dosen : Muhammad Reza Zulman, M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

2026

DAFTAR ISI

DAFTAR ISI ii

BAB I 1

PENDAHULUAN 1

- **A. Latar Belakang** 1
- **B. Tujuan Praktikum** 2

BAB II 3

LANGKAH KERJA OBJECT, CLASS & OOP 3

- **A. Persiapan Project** 3
- **B. Praktik: Mengukur Waktu Eksekusi Empiris** 3
- **C. Praktik: Contoh Tiap Kelas Big O** 5
- **D. Latihan 1: Identifikasi Kompleksitas** 8
- **E. Praktik: Perbandingan Space Complexity** 10
- **F. Latihan 2 : Class BankAccount** 11
- **G. Praktik: Refaktor Kode yang Lambat** 13
- **H. Tugas 1: Analisis dan Refactor** 14
- **I. Tugas 2: Visualisasi Pertembuhan Big O** 17
- **J. Submit ke GitHub** 20

BAB III 21

PENUTUP 21

- **A. Kesimpulan** 21
- **B. Saran** 21

BAB I

PENDAHULUAN

A. Latar Belakang

Dalam pengembangan perangkat lunak, efisiensi merupakan faktor krusial yang menentukan kualitas sebuah program. Sebuah algoritma tidak hanya dituntut untuk memberikan hasil yang benar, tetapi juga harus mampu bekerja dengan cepat dan menggunakan sumber daya memori yang minimal. Masalah sering muncul ketika suatu algoritma dihadapkan pada kumpulan data yang sangat besar; algoritma yang tampak efisien pada data kecil bisa menjadi sangat lambat saat skala input meningkat.

Analisis kompleksitas algoritma hadir sebagai metode matematis untuk mengukur kinerja algoritma berdasarkan dua aspek utama: kompleksitas waktu (time complexity) untuk mengukur kecepatan eksekusi, dan kompleksitas ruang (space complexity) untuk mengukur penggunaan memori. Dengan memahami notasi Big-O, pengembang dapat memprediksi perilaku algoritma dan memilih solusi yang paling optimal sebelum mengimplementasikannya dalam kode sumber. Praktikum ini dilakukan agar mahasiswa mampu menganalisis efisiensi berbagai algoritma secara sistematis.

.

B. Tujuan Praktikum

Tujuan dari praktikum Analisis Kompleksitas Algoritma ini adalah:

- Mahasiswa memahami konsep dasar kompleksitas algoritma dan notasi Big-O.
- Mahasiswa mampu menghitung dan menentukan kompleksitas waktu dari suatu algoritma.
- Mahasiswa mampu melakukan perbandingan efisiensi antara berbagai algoritma untuk menyelesaikan permasalahan yang sama.
- Mahasiswa mampu menganalisis pengaruh ukuran input terhadap waktu eksekusi program.

BAB II

LANGKAH KERJA OBJECT, CLASS & OOP

A. Persiapan Project

1. Buka terminal, navigasikan ke root folder repository kamu.
2. Buat folder praktikum-4/ dan masuk ke dalamnya.

```
mkdir praktikum-5
cd praktikum-5
```

3. Inisialisasi npm project.

```
npm init -y
```

B. Praktik: Mengukur Waktu Eksekusi Empiris

1. Pastikan kamu berada di dalam folder praktikum-5/. Buat file baru bernama 01-intro-kompleksitas.js.

```
touch 01-intro-kompleksitas.js
```

2. Buka file 01-intro-kompleksitas.js di VS Code, lalu ketikkan kode berikut.

```
function ukurWaktu(label, fn) {
  const mulai = Date.now();
  fn();
  const selesai = Date.now();
  console.log(`${label}: ${selesai - mulai} ms`);
}
```

```
const N = 100_000;

function jumlahkanLinear(n) {
  let total = 0;
  for (let i = 1; i <= n; i++) total += i;
  return total;
}

function jumlahkanRumus(n) {
  return (n * (n + 1)) / 2;
}

function cariPasangan(arr) {
  const pasangan = [];
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
    }
  }
  return pasangan;
}
const data = Array.from({ length: 5000 }, (_, i) => i);

console.log("=== Perbandingan Waktu Eksekusi ===");
ukurWaktu("0(1) jumlahkanRumus ", () => jumlahkanRumus(N));
ukurWaktu("0(n) jumlahkanLinear ", () => jumlahkanLinear(N));
ukurWaktu("0(n2) cariPasangan ", () => cariPasangan(data));

console.log("\nHasil sama?", jumlahkanLinear(100) ===
jumlahkanRumus(100));
```

3. Simpan file, lalu jalankan dari terminal.

```
node 01-intro-kompleksitas.js
```

```
USER@DESTYANASK MINGW64 /d/2025573010126_PraktikumStrukturData-Algoritma/praktikum-5 (main)
$ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 1 ms
O(n2) cariPasangan : 13 ms

Hasil sama? true
```

C. Praktik: Contoh Tiap Kelas Big O

1. Buat file baru bernama 02-kelas-big-o.js di dalam folder praktikum-5/.

```
touch 02-destructuring-spread.js
```

2. Buka file 02-kelas-big-o.js di VS Code, lalu ketikkan kode berikut.

```
function ambilPertama(arr) {
  return arr[0];
}
function tambahkanItem(arr, item) {
  arr.push(item);
}
function isGenap(n) {
  return n % 2 === 0;
}

function binarySearch(arr, target) {
  let kiri = 0,
    kanan = arr.length - 1;
  let langkah = 0;
  while (kiri <= kanan) {
    langkah++;
    const tengah = Math.floor((kiri + kanan) / 2);
    if (arr[tengah] === target) {
      console.log(` Ditemukan di indeks ${tengah} setelah
${langkah} langkah`);
      return tengah;
    }
    if (arr[tengah] < target) kiri = tengah + 1;
    else kanan = tengah - 1;
```

```
  }
  return -1;
}

function cariMax(arr) {
  let maks = arr[0];
  for (const x of arr) if (x > maks) maks = x;
  return maks;
}

function bubbleSortNaif(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length; i++)
    for (let j = 0; j < a.length - i - 1; j++) {
      if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]];
    }
  return a;
}

function fibRekursif(n) {
  if (n <= 1) return n;
  return fibRekursif(n - 1) + fibRekursif(n - 2);
}

console.log("=== O(1) - selalu cepat ===");
console.log(ambilPertama([10, 20, 30, 40, 50])); // 10
console.log(isGenap(42));

console.log("\n=== O(log n) - Binary Search ===");
const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
binarySearch(sorted, 731_452);

console.log("\n=== O(n) - Linear Search ===");
console.log(
  "Max dari 1000 elemen:",
  cariMax(Array.from({ length: 1000 }, () => (Math.random() *
1000) | 0)),
);

console.log("\n=== O(n2) - Bubble Sort (kecil saja) ===");
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));

console.log("\n=== O(2n) - Fibonacci Rekursif (n kecil!) ===");
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i)
+ " ");
```

```
console.log("");

console.log("\nWaktu fib(35) O(2n):");
let t = Date.now();
fibRekursif(35);
console.log(Date.now() - t, "ms");

console.log("Waktu fib(35) O(n) memoization:");
const memo = {};
function fibMemo(n) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  return (memo[n] = fibMemo(n - 1) + fibMemo(n - 2));
}
t = Date.now();
fibMemo(35);
console.log(Date.now() - t, "ms");
```

3. Simpan file, lalu jalankan dari terminal.

```
node 02-kelas-big-o.js
```

```
USER@DESTYANASK MINGW64 /d/2025573010126_PraktikumStrukturData-Algoritma/praktikum-5 (main)
$ node 02-kelas-big-o.js
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
 Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 999

=== O(n2) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2n) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2n):
70 ms
Waktu fib(35) O(n) memoization:
0 ms
```

D. Latihan 1: Identifikasi Kompleksitas

1. Buat file baru latihan-1.js
2. Isi code berikut yang telah dibuat berdasarkan permintaan Latihan 1 di modul

```
// --- Fungsi-fungsi dengan Big O ---

// O(1)
function fungsiA(n) {
    return n * 2;
}

// O(n^2)
function fungsiB(n) {
```

```
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            // operasi konstan
        }
    }
}

// O(log n)
function fungsiC(n) {
    for (let i = 1; i < n; i *= 2) {
        // operasi konstan
    }
}

// O(n^3) - Misal arr memiliki panjang n
function fungsiD(n) {
    const arr = new Array(n).fill(0);
    arr.forEach(x => arr.forEach(y => arr.forEach(z => {
        // operasi konstan
    })));
}

// --- Fungsi Pengukur Waktu ---

function hitungKompleksitas(n, fn) {
    const start = Date.now();
    fn(n);
    const end = Date.now();
    console.log(`Waktu eksekusi untuk n=${n}: ${end - start} ms`);
}

// --- Menjalankan Pengukuran ---

const n = 1000;

console.log("Pengukuran Waktu:");
hitungKompleksitas(n, fungsiA);
hitungKompleksitas(n, fungsiB);
hitungKompleksitas(n, fungsiC);
hitungKompleksitas(n, fungsiD);
```

3. Simpan file, lalu jalankan di terminal.

```
USER@DESTYANASK MINGW64 /d/2025573010126_PraktikumStrukturData-Algoritma/praktikum-5 (main)
$ node latihan-1.js
Pengukuran Waktu:
Waktu eksekusi untuk n=1000: 0 ms
Waktu eksekusi untuk n=1000: 2 ms
Waktu eksekusi untuk n=1000: 0 ms
Waktu eksekusi untuk n=1000: 460 ms
```

E. Praktik: Perbandingan Space Complexity

1. Buat file baru bernama 03-space-complexity.js di dalam folder praktikum-5/.
2. Buka file 03-space-complexity.js di VS Code, lalu ketikkan kode berikut.

```javascript
function jumlahArray(arr) {
  let total = 0;
  for (const x of arr) total += x;
  return total;
}

function duplikasiArray(arr) {
  const baru = [];
  for (const x of arr) baru.push(x * 2);
  return baru;
}

function faktorialRekursif(n) {
  if (n <= 1) return 1;
  return n * faktorialRekursif(n - 1);
}

function faktorialIteratif(n) {
  let hasil = 1;
  for (let i = 2; i <= n; i++) hasil *= i;
  return hasil;
}
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log("Jumlah array :", jumlahArray(arr));
console.log("Duplikasi array:", duplikasiArray(arr));
console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
console.log("Faktorial 10 iteratif :", faktorialIteratif(10));
```

```
function hitungUnik(arr) {
  const seen = new Set();
  for (const x of arr) seen.add(x);
  return seen.size;
}
const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
console.log("Elemen unik:", hitungUnik(dataAcak));
```

3. Simpan file, lalu jalankan dari terminal.

```
node 03-space-complexity.js
```

```
USER@DESTYANASK MINGW64 /d/2025573010126_PraktikumStrukturData-Algoritma/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
```

F. Latihan 2 : Class BankAccount

1. Tambahkan kode di bawah file 03-space-complexity.js.
2. Isi code berikut yang telah dibuat berdasarkan permintaan Latihan 2 di modul

```
console.log("LATIHAN");
// 1. Fungsi dengan Nested Loop (Brute Force)
function cariPasanganLambat(arr, target) {
  // Time Complexity: O(n^2) - Karena nested loop
  // Space Complexity: O(1) - Tidak menggunakan struktur data
tambahan
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
```

```
      }
    }
  }
  return null;
}

// 2. Fungsi dengan Set (Optimized)
function cariPasanganCepat(arr, target) {
  // Time Complexity: O(n) - Iterasi satu kali, lookup Set rata-
rata O(1)
  // Space Complexity: O(n) - Menyimpan elemen dalam Set
  const seen = new Set();
  for (const num of arr) {
    const complement = target - num;
    if (seen.has(complement)) {
      return [complement, num];
    }
    seen.add(num);
  }
  return null;
}

// 3. Uji Coba
const arrTest = [2, 7, 11, 15];
const target = 9;
console.log("Hasil Lambat:", cariPasanganLambat(arrTest, target));
// [2, 7]
console.log("Hasil Cepat:", cariPasanganCepat(arrTest, target));
// [2, 7]

// 4. Pengukuran Waktu dengan 50.000 data
const n = 50000;
const largeArr = Array.from({ length: n }, () =>
Math.floor(Math.random() * n));
const missingTarget = -1; // Target yang pasti tidak ada

function ukurWaktu(fn, arr, target) {
  const start = Date.now();
  fn(arr, target);
  console.log(`${fn.name} selesai dalam: ${Date.now() - start}
ms`);
}

ukurWaktu(cariPasanganLambat, largeArr, missingTarget);
ukurWaktu(cariPasanganCepat, largeArr, missingTarget);
```

G. Praktik: Refaktor Kode yang Lambat

1. Buat file baru bernama 04-refactor.js di dalam folder praktikum-5/.

```
touch 04-refactor.js
```

2. Buka file 04-refactor.js di VS Code, lalu ketikkan kode berikut.

```
function adaDuplikatLambat(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++) if (arr[i] ===
arr[j]) return true;
  return false;
}

function adaDuplikatCepat(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}

function frekuensiLambat(arr) {
  const unik = [];
  const hitung = [];
  for (const x of arr) {
    const idx = unik.indexOf(x);
    if (idx === -1) {
      unik.push(x);
      hitung.push(1);
    } else hitung[idx]++;
  }
  return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
}

function frekuensiCepat(arr) {
  const counter = {};
  for (const x of arr) counter[x] = (counter[x] || 0) + 1;
```

```
  return counter;
}

const besar = Array.from({ length: 20000 }, () =>
  Math.floor(Math.random() * 1000),
);
let t = Date.now();
adaDuplikatLambat(besar);
console.log("Duplikat LAMBAT O(n2):", Date.now() - t, "ms");
t = Date.now();
adaDuplikatCepat(besar);
console.log("Duplikat CEPAT O(n) :", Date.now() - t, "ms");
const kata = ["a", "b", "a", "c", "b", "a", "d"];
console.log("\nFrekuensi:", frekuensiCepat(kata));
```

3. Simpan file, lalu jalankan dari terminal.

```
node 04-refactor.js
```

```
USER@DESTYANASK MINGW64 /d/2025573010126_PraktikumStrukturData-Algoritma/praktikum-5 (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

H. Tugas 1: Analisis dan Refactor

1. Buat folder dan file tugas.

```
mkdir tugas

touch tugas/tugas-1.js tugas/tugas-2.js
```

2. Isi code berikut yang telah dibuat berdasarkan permintaan tugas 1 di modul

```
/**
 * TUGAS 1: ANALISIS DAN REFACTOR
```

```
 * File: tugas/tugas-1.js
 */

// --- 1. Fungsi A: Intersection ---
// O(n^2) - Nested loop
function intersectionLambat(arr1, arr2) {
  // Time: O(n*m), Space: O(min(n,m))
  let hasil = [];
  for (let i = 0; i < arr1.length; i++) {
    if (arr2.includes(arr1[i]) && !hasil.includes(arr1[i])) {
      hasil.push(arr1[i]);
    }
  }
  return hasil;
}

// O(n) - Menggunakan Set
function intersectionCepat(arr1, arr2) {
  // Time: O(n+m), Space: O(n)
  const set1 = new Set(arr1);
  return [...new Set(arr2.filter((item) => set1.has(item)))];
}

// --- 2. Fungsi B: Kelompok Anagram ---
function kelompokAnagram(strs) {
  // Time: O(n * k log k), Space: O(n * k) di mana k = panjang
kata
  const map = {};
  for (let str of strs) {
    let sorted = str.split("").sort().join("");
    if (!map[sorted]) map[sorted] = [];
    map[sorted].push(str);
  }
  return Object.values(map);
}

// --- 3. Fungsi C: Triplets (a + b = c^2) ---
// O(n^3) - Triple loop
function cekTripletLambat(arr) {
  // Time: O(n^3), Space: O(1)
  for (let i = 0; i < arr.length; i++) {
    for (let j = 0; j < arr.length; j++) {
      for (let k = 0; k < arr.length; k++) {
        if (i !== j && j !== k && i !== k && arr[i] + arr[j] ===
arr[k] ** 2) {
```

```
          return true;
        }
      }
    }
  }
  return false;
}

// O(n^2) - Menggunakan Set
function cekTripletCepat(arr) {
  // Time: O(n^2), Space: O(n)
  const set = new Set(arr);
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (set.has(Math.sqrt(arr[i] + arr[j]))) return true;
    }
  }
  return false;
}

// --- FUNGSI PENGUKUR WAKTU (BENCHMARK) ---
function benchmark(nama, fn, data) {
  const start = performance.now();
  fn(data);
  const end = performance.now();
  console.log(`${nama} selesai dalam: ${(end - start).toFixed(4)}
ms`);
}

// Contoh Pengujian
const dataBesar = Array.from({ length: 5000 }, (_, i) => i);
console.log("--- Pengujian Performa ---");
benchmark(
  "Intersection Cepat",
  () => intersectionCepat(dataBesar, dataBesar),
  null,
);
benchmark(
  "Anagram",
  () => kelompokAnagram(["eat", "tea", "tan", "ate", "nat",
"bat"]),
  null,
);
benchmark("Triplet Cepat", () => cekTripletCepat(dataBesar),
null);
```

3. Simpan dan jalankan: node tugas/tugas-1.js.

```
USER@DESTYANASK MINGW64 /d/2025573010126_PraktikumStrukturData-Algoritma/praktikum-5 (main)
$  node tugas/tugas-1.js
--- Pengujian Performa ---
Intersection Cepat selesai dalam: 0.6314 ms
Anagram selesai dalam: 0.0647 ms
Triplet Cepat selesai dalam: 0.8529 ms
```

I. Tugas 2: Visualisasi Pertembuhan Big O

1. Buat file tugas/tugas-2.js.
2. Isi code berikut yang telah dibuat berdasarkan permintaan tugas 2 di modul

```javascript
/**
 * TUGAS 2: VISUALISASI PERTUMBUHAN BIG O
 */

// 3. O(1)
function fn_01(n) {
  return n + 1;
}

// 4. O(n)
function fn_0n(n) {
  let sum = 0;
  for (let i = 0; i < n; i++) sum += i;
  return sum;
}

// 5. O(n log n)
function fn_0nLogn(n) {
  let count = 0;
  for (let i = 0; i < n; i++) {
    for (let j = 1; j < n; j *= 2) {
      count++;
    }
  }
  return count;
}
```

```
// 6. O(n^2)
function fn_0n2(n) {
  let count = 0;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      count++;
    }
  }
  return count;
}

// 2. Fungsi benchmarkSemua
function benchmarkSemua(ukuranData) {
  console.log("n\t | O(1) | O(n) | O(n log n) | O(n^2)");
  console.log("-----------------------------------------------
");

  for (let n of ukuranData) {
    const results = [];

    const fns = [fn_01, fn_0n, fn_0nLogn, fn_0n2];

    for (let fn of fns) {
      const start = performance.now();
      fn(n);
      const end = performance.now();
      results.push((end - start).toFixed(4));
    }

    console.log(`${n}\t | ${results.join("\t| ")}`);
  }
}

// 7. Panggilan
benchmarkSemua([100, 500, 1000, 5000, 10000]);

/**
 * 8. OBSERVASI:
 * - O(1): Waktu eksekusi sangat stabil dan hampir tidak berubah
meski n meningkat.
 * - O(n): Waktu meningkat secara linier (sebanding dengan n).
 * - O(n log n): Waktu meningkat sedikit lebih cepat dari linier.
 * - O(n^2): Waktu meningkat secara drastis (kuadratik). Saat
n=10.000,
 * perbedaannya dengan fungsi lain akan sangat terlihat jelas.
```

```
 * * Kesimpulan: Hasil observasi KONSISTEN dengan teori Big O, di
mana
 * fungsi dengan kompleksitas lebih tinggi akan jauh lebih lambat
 * saat n bertambah besar.
 */
```

3. Simpan dan jalankan: node tugas/tugas-2.js.

```
USER@DESTYANASK MINGW64 /d/2025573010126_PraktikumStrukturData-Algoritma/praktikum-5 (main)
$  node tugas/tugas-2.js
n        | O(1) | O(n) | O(n log n) | O(n^2)
-----------------------------------------------
100      | 0.0069       | 0.0137       | 0.0177       | 0.1587
500      | 0.0013       | 0.0028       | 0.0459       | 0.4202
1000     | 0.0005       | 0.0204       | 0.0267       | 0.9457
5000     | 0.0004       | 0.0037       | 0.0539       | 17.1264
10000    | 0.0005       | 0.0099       | 0.1406       | 68.2135
```

J. Submit ke GitHub

1. Kembali ke root folder repository.

```
cd ..
git status
```

2. Tambahkan folder praktikum-5/ ke staging.

```
git add praktikum-5/
```

3. Commit dan push.

```
git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
git push origin main
```

BAB III

PENUTUP

A. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Analisis kompleksitas algoritma sangat penting untuk menentukan efisiensi algoritma tanpa harus bergantung pada spesifikasi perangkat keras.
2. Notasi Big-O memberikan batas atas (upper bound) dari pertumbuhan waktu eksekusi sebuah algoritma seiring dengan meningkatnya jumlah data input (n).

B. Saran

Untuk pengembangan pemahaman materi di masa mendatang, disarankan:

- Agar lebih memahami implementasi, mahasiswa sebaiknya mencoba membandingkan algoritma secara empiris (mengukur waktu eksekusi sebenarnya) dan membandingkannya dengan analisis teoretis Big-O.
- Perlunya ketelitian lebih lanjut dalam menghitung operasi dominan dalam sebuah kode program agar hasil analisis kompleksitas lebih akurat.
- Mahasiswa diharapkan terus mengeksplorasi struktur data yang berbeda agar dapat mengoptimalkan pemilihan algoritma yang tepat untuk kasus yang spesifik.